

Classes and Objects

CSCI 1101

2024-04-11

Turtle Setup

We're going to do some animation with `turtle` today:

```
import turtle
```

```
def init_turtle() :  
    # don't show the turtle icon:  
    turtle.hideturtle()  
    # default turtle state is not drawing:  
    turtle.penup()  
    # don't render to screen unless instructed:  
    turtle.tracer(0 , 0)
```

Drawing a Dot

Suppose that we want turtle to draw a dot at a given point:

```
dot_size = 10
```

```
def dot_at(coords) :  
    (x , y) = coords  
    turtle.goto(x , y)  
    turtle.dot(dot_size)
```

We can specify a dot with a given position:

```
dot_pos = (0 , 0)
```

And write a function to draw that dot at its current location:

```
def draw_dot() :  
    init_turtle()  
    dot_at(dot_pos)
```

Moving the Dot – First Attempt

An **event handler** is a function that we **register** with turtle.

Turtle will call the event handler function whenever the specified event occurs.

Let's try to add an event handler to move the dot.

```
def move_dot_right() :  
    turtle.clear() # erase old scene  
    dot_pos = (dot_pos[0] + 10 , dot_pos[1])  
    dot_at(dot_pos)  
    turtle.update() # paint new scene  
  
def run_dot() :  
    init_turtle()  
    dot_at(dot_pos)  
    turtle.onkeypress(move_dot_right , 'Right')  
    turtle.listen()
```

Moving the Dot – Failure Analysis

This fails because `dot_pos` is a **global variable**, so we can't reassign it from within a function. And because `tuples` are immutable, we can't update its value either.

It won't work to make `dot_pos` a **local variable** of the `move_dot_right` callback function, because then it would go out of scope when the function returns and the new position would be lost.

If `dot_pos` were an object of a **mutable type** then `move_dot_right` could update its state and those changes would persist after the function call ends.

Classes and Objects

One solution to this problem is to define a new custom mutable type, called as a **class**:

```
class Point : # class header
    # our class doesn't do anything yet
    pass
```

Then we can make a new element of this type, called a class instance **object**:

```
my_point = Point()
```

Each object has its own private variable namespace for storing its **state**:

```
my_point.x = 0
my_point.y = 0
```

These private variables of an object are called its **attributes** or **fields**.

An Interface for Points

Instead of manipulating the state of a `Point` directly, we can use an **interface**.

This is a collection of functions that lets us interact with its state in a controlled way.

```
def set_pos(point , position) :  
    ''' (Point , tuple (int , int)) --> NoneType '''  
    (point.x , point.y) = position  
  
def get_pos(point) :  
    ''' (Point) --> tuple (int , int) '''  
    return (point.x , point.y)  
  
def move_right(point) :  
    ''' (Point) --> NoneType '''  
    point.x += 10
```

Moving the Dot – Second Attempt

Now we can use a mutable `Point` object in the event handler for moving the dot:

```
def move_dot_right() :  
    turtle.clear() # erase old scene  
    move_right(my_point)  
    dot_at(get_pos(my_point))  
    turtle.update() # paint new scene  
  
def run_dot () :  
    init_turtle()  
    dot_at(get_pos(my_point))  
    turtle.onkeypress(move_dot_right , 'Right')  
    turtle.listen()
```


Moving the Interface into the Class

Recall that we can use *method syntax* for functions that act on certain types.

This will not work with our `Point` type yet.

Moving the interface functions into the class definition makes them **methods**:

```
class Point :  
    def set_pos (point , position) :  
        (point.x , point.y) = position  
  
    def get_pos(point) :  
        return (point.x , point.y)  
  
    def move_right (point) :  
        point.x += 10
```

Now they are no longer global functions and we can call them using method syntax.

Initializing Objects

It's annoying that before we can use a `Point` object, we have to manually initialize it by calling `set_pos`.

We can make this initialization step part of the object-creation process by defining an **initializer method** with the special name “`__init__`”.

The first argument of the `__init__` method is a newly-created object of its class; it can take any additional arguments arguments you want.

The initializer is automatically called whenever you create a new object of the class. It is conventional (but optional) to name the first argument of a method “`self`”.

The Point Class – Final Version

Here is the final version of our Point class:

```
class Point :  
    def __init__(self , init_x , init_y) :  
        self.x = init_x  
        self.y = init_y  
  
    def set_pos(self , position) :  
        (self.x , self.y) = position  
  
    def get_pos(self) :  
        return (self.x , self.y)  
  
    def move_right(self) :  
        self.x += 10
```

Moving Dot – Final Version

Here is the final version of our dot-moving program:

```
my_point = Point(0 , 0)
```

```
def move_dot_right() :  
    turtle.clear() # erase old scene  
    my_point.move_right()  
    dot_at(my_point.get_pos())  
    turtle.update() # paint new scene  
  
def run_dot() :  
    init_turtle()  
    dot_at(my_point.get_pos())  
    turtle.onkeypress(move_dot_right , 'Right')  
    turtle.listen()
```

Activity: Moving Left

Augment the `Point` class and the dot-moving program so that the dot can move to the left as well as to the right.

To Do

Finish *Homework 10: Object-Oriented Programming* on CodeRunner by 11:00AM next Thursday.

Finish on *Project 3 - Automated Grader (Part 2)* on CodeRunner by 12:00PM next Friday.